ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 3

INTRODUCTION TO TECHNICAL PROGRAMMING

PRACTICE EXAM 6 (PAPER 2)

MARKS: 100 **TIME: 3 hours**

NC(V) Programming and Robotics, Level 3. Laid out like the DHET second paper (practical, 100 marks): an mBlock taxi-fare app, a C++ robot wheel-speed checker with selection, and a C++ rainfall program with arrays, each with an IPO chart.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 3. It is not a Department of Higher Education and Training examination paper.

INSTRUCTIONS AND INFORMATION

1. This question paper consists of THREE questions. Answer ALL the questions.
2. This is a practical paper. Use mBlock for QUESTION 1 and a C++ development environment (such as Geany or Code::Blocks) for QUESTIONS 2 and 3. If you do not have a computer, write the programs neatly in your answer book.
3. Save each program with the question number and your name or examination number, and add your name or examination number as a comment on the first line of each C++ program.
4. Save your work regularly.
5. Programs are not marked strictly on syntax.

QUESTION 1

> A taxi association wants a simple mBlock app that works out fares. For each trip the sprite asks for the distance in kilometres and the number of passengers. The fare is R12 plus R8 for every kilometre. If there are more than 4 passengers, R10 is added for the bigger vehicle. The sprite says the fare of each trip. The app handles 3 trips and then says the total of all the fares.

Use the IPO chart below to help you complete the application.

INPUT	PROCESS	OUTPUT
Distance (km) Number of passengers	Declare all variables and set them to 0 Repeat 3 times: Ask for the distance and the passengers fare = 12 + 8 × distance If passengers > 4 then fare = fare + 10 Say the fare total = total + fare Say the total	Fare of each trip Total of all fares

Follow the instructions below to create the mBlock application:

1.1 Open mBlock, create a new project and name it Question1_YourName. (1)

1.2 Choose a sprite to act as the taxi rank marshal. (1)

1.3 Choose a backdrop that shows a street or a taxi rank. (1)

1.4 Declare all the variables and give them their starting values. (8)

1.5 Use a loop that repeats for 3 trips. (2)

1.6 Ask for the distance and store the answer. (3)

1.7 Ask for the number of passengers and store the answer. (3)

1.8 Calculate the fare. (3)

1.9 Add the extra R10 when there are more than 4 passengers. (4)

1.10 Display the fare of the trip. (2)

1.11 Update the total. (1)

1.12 Display the total after the 3 trips. (1)

1.13 Test the program with these trips and write down the fares and total it displays: 5 km with 2 passengers; 10 km with 6 passengers; 3 km with 1 passenger. (2)

[32]

QUESTION 2

A robotics club wants a program that works out how fast its robot moves. The user enters the motor speed in revolutions per minute (rpm) and the wheel diameter in metres. The program calculates the robot's speed and shows a speed category.

Speed formula: speed (m/s) = π × diameter × rpm ÷ 60 (use 3.14159 for π)

Speed categories: Slow – less than 0,5 m/s; Normal – 0,5 m/s up to (but not including) 1,5 m/s; Fast – 1,5 m/s or more.

INPUT	PROCESS	OUTPUT
rpm diameter	Declare all variables and set them to 0 Ask for the rpm and the diameter speed = 3.14159 × diameter × rpm ÷ 60 If speed < 0,5: category = "Slow" Else if speed < 1,5: category = "Normal" Else: category = "Fast"	speed category

2.1 Create a new C++ source file called Question2_YourName.cpp. (1)

2.2 Include the necessary header(s) and use the std namespace. (2)

2.3 Define the main function and declare all variables, setting them to 0 (or an empty string). (4)

2.4 Ask the user for the rpm and the wheel diameter and store them in the correct variables. (4)

2.5 Use the formula to calculate the speed. (3)

2.6 Use if, else if and else to find the speed category. (8)

2.7 Display the speed (in m/s) and the category. (3)

[25]

QUESTION 3

A farmer records the rainfall on her farm every day. She wants a C++ program that stores the rainfall for a number of days (at most 31) and then shows the total rainfall, the average, the wettest day and the number of dry days (days with 0 mm).

INPUT	PROCESS	OUTPUT
Number of days Rainfall for each day (mm)	Declare all variables Ask for the number of days (repeat until it is from 1 to 31) Store the rainfall of each day in an array	Total rainfall Average rainfall Wettest day and its

	total = total + rain[i] average = total ÷ number of days Find the largest rainfall and its day number Count the days with 0 mm	rainfall Number of dry days

The following steps show how the program should be used:

```
Enter the number of days (1-31): 40
Enter the number of days (1-31): 5
Rainfall for day 1 (mm): 0
Rainfall for day 2 (mm): 12.5
Rainfall for day 3 (mm): 4
Rainfall for day 4 (mm): 0
Rainfall for day 5 (mm): 21
Total rainfall: 37.5 mm
Average rainfall: 7.5 mm per day
Wettest day: day 5 with 21 mm
Dry days: 2
```

3.1 Create a new C++ source file called Question3_YourName.cpp. (1)

3.2 Include the necessary header(s) and use the std namespace. (2)

3.3 Define the main function and declare the variables. (4)

3.4 Ask for the number of days, and ask again until the number is from 1 to 31. (4)

3.5 Declare the array that stores the daily rainfall. (4)

3.6 Use a loop to ask for and store the rainfall of each day. (6)

3.7 Calculate the total rainfall. (4)

3.8 Find the largest rainfall and the number of the day on which it fell. (8)

3.9 Count the number of dry days. (4)

3.10 Calculate the average rainfall. (2)

3.11 Display the total, the average, the wettest day and the number of dry days. (4)

[43]

TOTAL: 100

END OF QUESTION PAPER